

```
In [1]: # Behzad Ghabaei
# CS82A - Data Science
# Text book Problem 2.2
# Bridges in Pittsburgh
# module3_Lab3.ipynb
# Instructor Seno
# 9/17/2026
```

Step 1: Load the dataset

The 4 commands you run on a new dataset are: `df.head()` `df.shape` `df.info()` `df.describe()`.

```
In [2]: import pandas as pd
df = pd.read_csv("bridges.csv")
```

```
In [3]: df.shape
```

Out[3]: (108, 13)

```
In [4]: df.head(5)
```

Out[4]:

	Id	river	location	erected	purpose	length	lanes	clear- g	t-or-d	material	span	rel- l	type
0	E1	M	3.0	1818	HIGHWAY	NaN	2.0	N	THROUGH	WOOD	SHORT	S	WOOD
1	E2	A	25.0	1819	HIGHWAY	1037.0	2.0	N	THROUGH	WOOD	SHORT	S	WOOD
2	E3	A	39.0	1829	AQUEDUCT	NaN	1.0	N	THROUGH	WOOD	NaN	S	WOOD
3	E5	A	29.0	1837	HIGHWAY	1000.0	2.0	N	THROUGH	WOOD	SHORT	S	WOOD
4	E6	M	23.0	1838	HIGHWAY	NaN	2.0	N	THROUGH	WOOD	NaN	S	WOOD

```
In [5]: df.tail(5)
```

Out[5]:

	Id	river	location	erected	purpose	length	lanes	clear- g	t-or-d	material	span	rel- l	tyf
103	E84	A	24.0	1969	HIGHWAY	870.0	6.0	G	THROUGH	STEEL	MEDIUM	F	ARC
104	E91	O	44.0	1975	HIGHWAY	3756.0	6.0	G	THROUGH	STEEL	LONG	F	ARC
105	E90	M	7.0	1978	HIGHWAY	950.0	6.0	G	THROUGH	STEEL	LONG	F	ARC
106	E100	O	43.0	1982	HIGHWAY	NaN	NaN	G	NaN	NaN	NaN	F	Na
107	E109	A	28.0	1986	HIGHWAY	NaN	NaN	G	NaN	NaN	NaN	F	Na



Next, lets take a look at the data types

```
In [6]: df.dtypes
```

```
Out[6]: Id          str
        river       str
        location    float64
        erected     int64
        purpose     str
        length      float64
        lanes       float64
        clear-g     str
        t-or-d      str
        material    str
        span        str
        rel-l       str
        type        str
        dtype: object
```

It could be better to change the data type of lanes to int. This could prevent noisy data, such as saying two lanes can be expressed as 2.0000. Next lets look at null values or missing values in the dataset.

```
In [7]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 108 entries, 0 to 107
Data columns (total 13 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Id          108 non-null   str
1   river       108 non-null   str
2   location    107 non-null   float64
3   erected     108 non-null   int64
4   purpose     108 non-null   str
5   length      81 non-null    float64
6   lanes       92 non-null    float64
7   clear-g     106 non-null   str
8   t-or-d      102 non-null   str
9   material    106 non-null   str
10  span        92 non-null    str
11  rel-l       103 non-null   str
12  type        106 non-null   str
dtypes: float64(3), int64(1), str(9)
memory usage: 11.1 KB
```

From looking at `df.shape` we can see there are 108 rows, so the columns requiring imputation are: **location**, **length**, **lanes**, **clear-g**, **t-or-d**, **material**, **span**, **rel-l**, and **type**. Lets learn the spelling of our columns.

```
In [8]: df.columns
```

```
Out[8]: Index(['Id', 'river', 'location', 'erected', 'purpose', 'length', 'lanes',
              'clear-g', 't-or-d', 'material', 'span', 'rel-l', 'type'],
              dtype='str')
```

Lets look at the median for: length, lanes, and location.

```
In [9]: df.describe()
```

Out[9]:

	location	erected	length	lanes
count	107.000000	108.000000	81.000000	92.000000
mean	25.978505	1905.314815	1567.469136	2.630435
std	13.665871	37.173664	747.491523	1.164657
min	1.000000	1818.000000	804.000000	1.000000
25%	15.500000	1884.000000	1000.000000	2.000000
50%	27.000000	1903.000000	1300.000000	2.000000
75%	37.500000	1928.000000	2000.000000	4.000000
max	52.000000	1986.000000	4558.000000	6.000000

Step 2: Write your first observations

Lets start from the first column, "Id" - we see inconsistencies such as E28 jumps to E32. Aso E4 next to E40, and E89. Is this naming convention the actual posted name of the bridge, or can it be changed? They are numbered, but not sorted. But this is fine because the Id could represent famous names for bridges.

Next we see "river" is M or A or O or Y. Under Pittsburgh Bridges, we see: Monongahela River, Allegheny River, Ohio river, and Youghiogheny.

"location" can be studied in the describe function. Count: 107, signifies one is missing. the min is 1 and the maz is 52. The location should be precise and an approximation does help. If we say it's 52, this means far away, if we say 1 it signifies close by. We can use the median for this missing value which will not skew the data much.

"erected" is straight forward and signifies when it was built, this is an integer that can be sorted and not missing any data. What's nice is that the min shows the earliest date at 1818 and the max shows us most recent 1986.

"purpose" listed "HIGHWAY", "AQUEDUCT", "RR", and a single "WALK" entry.

"length" is a float and is missing 81 - 108 values. The nice thing is that the median can be obtained from the descibe() function. We could say this is in feet, or if there are too many extra zeros, we can adjust the data and we can use the median for the missing values.

"lanes" is missing some values, 92 counted values non - null out of 108, and this is simply done with an integer. It look like 1 or 2, but we can check to see how many actually are more than 2 to be sure and control this from listing an impossible number. The majority of lanes were probably 2, 4 and 6, which is most recent and pulls up the average.

The remainder of the data is in str format, so individually inserting data, can be the best method to pre-process this data.

"clear-g" is either N or G. Mostly N from the 1800s and mostly G after the civil war. Missing a few values 106 - 108. Can be inserted as a string.

"t-or-d" indicated THROUGH or DECK. The majority are THROUGH but some are DECK. How can we fill in the missing string values, by using THROUGH instead of DECK. DECK is much more scarce.

"span" is indicated with SHORT, MEDIUM, or LONG, and is not related to the length. It holds a relationship with number of lanes, A string is used and this column is missing many entries with 92 - 108.

"rel-l" shows us S, F, S-F, missing 103 - 108. Meaning that 5 need to be filled.

"type" is the last column and indicated "WOOD", "SUSPEN", SIMPLE-T, ARCH, CANTILEV, NIL, CONT-T. We can tell that wood is in the past from the 1800s. But the non null counts up to 106 and this means a few need to be addressed.

Step 3: Count the missing values

```
In [10]: # apply tutorial 1 to the whole dataframe
df.isna().sum()
```

```
Out[10]: Id          0
river          0
location       1
erected        0
purpose        0
length        27
lanes          16
clear-g        2
t-or-d         6
material       2
span          16
rel-l          5
type           2
dtype: int64
```

This is a very helpful list of missing values.

Step 4: Impute the missing entries

```
In [11]: df.shape # always run shape before and after imputing
```

```
Out[11]: (108, 13)
```

Lets take a look at the data types again to insert our new values.

```
In [12]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 108 entries, 0 to 107
Data columns (total 13 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Id          108 non-null   str
1   river       108 non-null   str
2   location    107 non-null   float64
3   erected     108 non-null   int64
4   purpose     108 non-null   str
5   length      81 non-null    float64
6   lanes       92 non-null    float64
7   clear-g     106 non-null   str
8   t-or-d      102 non-null   str
9   material    106 non-null   str
10  span        92 non-null    str
11  rel-l       103 non-null   str
12  type        106 non-null   str
dtypes: float64(3), int64(1), str(9)
memory usage: 11.1 KB
```

```
In [13]: # Fill the missing - Tutorial 2.
# Looking for: Impossible Values, Suspicious Outliers, Missing Values, Floating Point Noise
```

```
fill_value = df["location"].median() # The 50% median
df["location"] = df["location"].fillna(fill_value)
```

```
In [14]: df.shape
```

```
Out[14]: (108, 13)
```

Check to see if it works, under location should be at 0.

```
In [15]: df.isna().sum()
```

```
Out[15]: Id          0
         river       0
         location    0
         erected     0
         purpose     0
         length     27
         lanes      16
         clear-g     2
         t-or-d      6
         material    2
         span       16
         rel-l       5
         type        2
         dtype: int64
```

We could do this for "length" and "lanes", then, remove duplicate rows and then, convert the data type of "lanes".

```
In [16]: fill_value = df["length"].median() # The 50% median
         df["length"] = df["length"].fillna(fill_value)
```

Lanes are normally even numbers that double after a one way road is addressed, so the median should be observed, and rounded to an even number divisible by 2

```
In [17]: print("median of lanes is", df["lanes"].median())
```

```
median of lanes is 2.0
```

We can now use the median for lanes, however in recent times 6 is most prominent and 2 of the missing values are modern era.

```
In [18]: fill_value = df["lanes"].median() # The 50% median
         df["lanes"] = df["lanes"].fillna(fill_value)
         df.loc[106, "lanes"] = 6
         df.loc[107, "lanes"] = 6
```

```
In [19]: df.shape
```

```
Out[19]: (108, 13)
```

```
In [20]: # apply tutorial 1 to the whole dataframe
         df.isna().sum()
```

```
Out[20]: Id          0
         river      0
         location   0
         erected    0
         purpose    0
         length     0
         lanes      0
         clear-g    2
         t-or-d     6
         material    2
         span      16
         rel-l      5
         type       2
         dtype: int64
```

```
In [21]: df.shape
```

```
Out[21]: (108, 13)
```

Since there are two entries needed which divide clear-g into N and G, we could make the first one "N" and the second one "G", to even it out. *Important datasets from csv files do not have the same index as .isna() for filling individual strings.

```
In [22]: df[df["clear-g"].isna()]
```

```
Out[22]:
```

	Id	river	location	erected	purpose	length	lanes	clear-g	t-or-d	material	span	rel-l	type
19	E21	M	16.0	1874	RR	1300.0	2.0	NaN	THROUGH	IRON	NaN	NaN	SIMPLE-T
20	E23	M	1.0	1876	HIGHWAY	1245.0	2.0	NaN	THROUGH	STEEL	LONG	F	SUSPEN

```
In [23]: # clear-g is missing 2
         df.loc[19, "clear-g"] = "N"
         df.loc[20, "clear-g"] = "G"
```

```
In [24]: df.shape
```

```
Out[24]: (108, 13)
```

Since the majority of entries of "t-or-d" are THROUGH, fill the missing entries with THROUGH. Since material is missing 2 entries from the years 1982 and 1986, we can estimate the most recent material to be STEEL. For span we can study the relationship with length and lanes to decide SHORT, MEDIUM, LONG.

```
In [25]: #t-or-d is missing 6
         fill_value = "THROUGH"
         df["t-or-d"] = df["t-or-d"].fillna(fill_value)
```

```
In [26]: # material is missing 2
         fill_value = "STEEL"
         df["material"] = df["material"].fillna(fill_value)
```

It may be useful to flag these 16 entries from span with a new column called span_flag

```
In [27]: df[df["span"].isna()]
```

Out[27]:

	ld	river	location	erected	purpose	length	lanes	clear- g	t-or-d	material	span	rel-l	ty
2	E3	A	39.0	1829	AQUEDUCT	1300.0	1.0	N	THROUGH	WOOD	NaN	S	WO
4	E6	M	23.0	1838	HIGHWAY	1300.0	2.0	N	THROUGH	WOOD	NaN	S	WO
8	E10	A	39.0	1848	AQUEDUCT	1300.0	1.0	N	DECK	WOOD	NaN	S	WO
10	E12	A	39.0	1853	RR	1300.0	2.0	N	DECK	WOOD	NaN	S	WO
12	E13	A	33.0	1856	HIGHWAY	1300.0	2.0	N	THROUGH	WOOD	NaN	S	WO
13	E15	A	28.0	1857	RR	1300.0	2.0	N	THROUGH	WOOD	NaN	S	WO
19	E21	M	16.0	1874	RR	1300.0	2.0	N	THROUGH	IRON	NaN	NaN	SIMP
22	E24	O	45.0	1878	RR	1300.0	2.0	G	THROUGH	STEEL	NaN	NaN	SIMP
23	E25	M	10.0	1882	RR	1300.0	2.0	G	THROUGH	STEEL	NaN	NaN	SIMP
24	E27	A	39.0	1883	RR	1300.0	2.0	G	THROUGH	STEEL	NaN	F	SIMP
45	E45	M	14.0	1897	RR	2264.0	2.0	G	THROUGH	STEEL	NaN	F	SIMP
63	E107	A	39.0	1914	RR	1300.0	2.0	G	THROUGH	STEEL	NaN	F	I
69	E65	A	30.0	1921	WALK	1300.0	2.0	G	THROUGH	STEEL	NaN	F	SUSP
98	E82	O	42.0	1955	HIGHWAY	804.0	2.0	G	THROUGH	STEEL	NaN	F	SIMP
106	E100	O	43.0	1982	HIGHWAY	1300.0	6.0	G	THROUGH	STEEL	NaN	F	N
107	E109	A	28.0	1986	HIGHWAY	1300.0	6.0	G	THROUGH	STEEL	NaN	F	N

```
In [28]: def span_condition(v):
        if v == "LONG" or v == "SHORT" or v == "MEDIUM":
            return "no flag"
        else:
            return "flag"
        # the apply function was needed here
        df['span_flag'] = df["span"].apply(span_condition)
        df.head(60)
```

Out[28]:

	Id	river	location	erected	purpose	length	lanes	clear- g	t-or-d	material	span	rel-l	
0	E1	M	3.0	1818	HIGHWAY	1300.0	2.0	N	THROUGH	WOOD	SHORT	S	V
1	E2	A	25.0	1819	HIGHWAY	1037.0	2.0	N	THROUGH	WOOD	SHORT	S	V
2	E3	A	39.0	1829	AQUEDUCT	1300.0	1.0	N	THROUGH	WOOD	NaN	S	V
3	E5	A	29.0	1837	HIGHWAY	1000.0	2.0	N	THROUGH	WOOD	SHORT	S	V
4	E6	M	23.0	1838	HIGHWAY	1300.0	2.0	N	THROUGH	WOOD	NaN	S	V
5	E7	A	27.0	1840	HIGHWAY	990.0	2.0	N	THROUGH	WOOD	MEDIUM	S	V
6	E8	A	28.0	1844	AQUEDUCT	1000.0	1.0	N	THROUGH	IRON	SHORT	S	SL
7	E9	M	3.0	1846	HIGHWAY	1500.0	2.0	N	THROUGH	IRON	SHORT	S	SL
8	E10	A	39.0	1848	AQUEDUCT	1300.0	1.0	N	DECK	WOOD	NaN	S	V
9	E11	A	29.0	1851	HIGHWAY	1000.0	2.0	N	THROUGH	WOOD	MEDIUM	S	V
10	E12	A	39.0	1853	RR	1300.0	2.0	N	DECK	WOOD	NaN	S	V
11	E14	M	6.0	1856	HIGHWAY	1200.0	2.0	N	THROUGH	WOOD	MEDIUM	S	V
12	E13	A	33.0	1856	HIGHWAY	1300.0	2.0	N	THROUGH	WOOD	NaN	S	V
13	E15	A	28.0	1857	RR	1300.0	2.0	N	THROUGH	WOOD	NaN	S	V
14	E16	A	25.0	1859	HIGHWAY	1030.0	2.0	N	THROUGH	IRON	MEDIUM	S-F	SL
15	E17	M	4.0	1863	RR	1000.0	2.0	N	THROUGH	IRON	MEDIUM	NaN	SIM
16	E18	A	28.0	1864	RR	1200.0	2.0	N	THROUGH	IRON	SHORT	S	SIM
17	E19	A	29.0	1866	HIGHWAY	1000.0	2.0	N	THROUGH	WOOD	MEDIUM	S	V
18	E20	A	32.0	1870	HIGHWAY	1000.0	2.0	N	THROUGH	WOOD	MEDIUM	S	V
19	E21	M	16.0	1874	RR	1300.0	2.0	N	THROUGH	IRON	NaN	NaN	SIM
20	E23	M	1.0	1876	HIGHWAY	1245.0	2.0	G	THROUGH	STEEL	LONG	F	SL
21	E22	A	24.0	1876	HIGHWAY	1200.0	4.0	G	THROUGH	WOOD	SHORT	S	V
22	E24	O	45.0	1878	RR	1300.0	2.0	G	THROUGH	STEEL	NaN	NaN	SIM
23	E25	M	10.0	1882	RR	1300.0	2.0	G	THROUGH	STEEL	NaN	NaN	SIM
24	E27	A	39.0	1883	RR	1300.0	2.0	G	THROUGH	STEEL	NaN	F	SIM
25	E26	M	12.0	1883	RR	1150.0	2.0	G	THROUGH	STEEL	MEDIUM	S	SIM
26	E30	A	31.0	1884	RR	1300.0	2.0	G	THROUGH	STEEL	MEDIUM	F	SIM
27	E29	A	26.0	1884	HIGHWAY	1080.0	2.0	G	THROUGH	STEEL	MEDIUM	NaN	SL
28	E28	M	3.0	1884	HIGHWAY	1000.0	2.0	G	THROUGH	STEEL	MEDIUM	S	
29	E32	A	30.0	1887	HIGHWAY	1300.0	2.0	G	THROUGH	IRON	MEDIUM	F	SIM
30	E31	M	8.0	1887	RR	1161.0	2.0	G	THROUGH	STEEL	MEDIUM	S	SIM
31	E34	O	41.0	1888	RR	4558.0	2.0	G	THROUGH	STEEL	LONG	F	SIM
32	E33	M	19.0	1889	HIGHWAY	1120.0	2.0	G	THROUGH	IRON	MEDIUM	F	SIM
33	E36	O	45.0	1890	HIGHWAY	1300.0	2.0	G	THROUGH	IRON	SHORT	F	SIM
34	E35	A	27.0	1890	HIGHWAY	1000.0	2.0	G	THROUGH	STEEL	MEDIUM	F	SIM

	Id	river	location	erected	purpose	length	lanes	clear- g	t-or-d	material	span	rel-l	
35	E38	M	17.0	1891	HIGHWAY	1300.0	2.0	G	THROUGH	IRON	MEDIUM	F	SIM
36	E37	M	18.0	1891	RR	1350.0	2.0	G	THROUGH	STEEL	MEDIUM	S	SIM
37	E39	A	25.0	1892	HIGHWAY	1300.0	2.0	G	THROUGH	STEEL	MEDIUM	F	SIM
38	E4	A	27.0	1892	AQUEDUCT	1092.0	1.0	N	THROUGH	WOOD	SHORT	S	V
39	E40	M	22.0	1893	HIGHWAY	1300.0	2.0	G	THROUGH	STEEL	MEDIUM	F	SIM
40	E41	M	11.0	1894	HIGHWAY	1300.0	2.0	G	THROUGH	IRON	MEDIUM	F	SIM
41	E42	M	9.0	1895	HIGHWAY	2367.0	2.0	G	THROUGH	STEEL	LONG	F	SIM
42	E44	O	48.0	1896	HIGHWAY	1300.0	2.0	G	THROUGH	STEEL	LONG	F	SL
43	E43	M	7.0	1896	HIGHWAY	1040.0	2.0	G	THROUGH	STEEL	LONG	F	
44	E46	A	37.0	1897	RR	4000.0	2.0	G	DECK	STEEL	LONG	F	SIM
45	E45	M	14.0	1897	RR	2264.0	2.0	G	THROUGH	STEEL	NaN	F	SIM
46	E47	M	15.0	1898	RR	2000.0	2.0	G	THROUGH	STEEL	MEDIUM	S	SIM
47	E58	A	33.0	1900	HIGHWAY	1200.0	2.0	G	THROUGH	STEEL	MEDIUM	F	SIM
48	E48	A	38.0	1900	HIGHWAY	2000.0	2.0	G	THROUGH	STEEL	MEDIUM	F	SIM
49	E94	M	13.0	1901	RR	1300.0	2.0	G	THROUGH	STEEL	LONG	F	SIM
50	E49	A	34.0	1902	HIGHWAY	1850.0	2.0	G	THROUGH	STEEL	MEDIUM	F	CAN
51	E95	M	16.0	1903	RR	1300.0	2.0	G	THROUGH	STEEL	MEDIUM	S	SIM
52	E87	A	35.0	1903	RR	3000.0	2.0	G	THROUGH	STEEL	MEDIUM	S	SIM
53	E51	M	6.0	1903	RR	1417.0	2.0	G	THROUGH	STEEL	MEDIUM	F	SIM
54	E50	M	21.0	1903	RR	1154.0	2.0	G	THROUGH	STEEL	LONG	F	SIM
55	E89	M	4.0	1904	RR	1200.0	2.0	G	THROUGH	STEEL	MEDIUM	S-F	SIM
56	E53	A	28.0	1904	RR	965.0	4.0	G	THROUGH	STEEL	MEDIUM	S-F	SIM
57	E52	M	2.0	1904	RR	1504.0	2.0	G	THROUGH	STEEL	LONG	F	CAN
58	E54	Y	27.0	1908	HIGHWAY	1240.0	2.0	G	THROUGH	STEEL	MEDIUM	F	SIM
59	E56	M	23.0	1909	HIGHWAY	1300.0	2.0	G	THROUGH	STEEL	MEDIUM	F	SIM

```
In [29]: def t_or_d_condition(v):
          if v == "THROUGH" or v == "DECK":
              return "no flag"
          else:
              return "flag"
          # the apply function was needed here
          df['t_or_d_flag'] = df["t-or-d"].apply(t_or_d_condition)
          df.head(10)
```

Out[29]:

	Id	river	location	erected	purpose	length	lanes	clear- g	t-or-d	material	span	rel- l	typ
0	E1	M	3.0	1818	HIGHWAY	1300.0	2.0	N	THROUGH	WOOD	SHORT	S	WOC
1	E2	A	25.0	1819	HIGHWAY	1037.0	2.0	N	THROUGH	WOOD	SHORT	S	WOC
2	E3	A	39.0	1829	AQUEDUCT	1300.0	1.0	N	THROUGH	WOOD	NaN	S	WOC
3	E5	A	29.0	1837	HIGHWAY	1000.0	2.0	N	THROUGH	WOOD	SHORT	S	WOC
4	E6	M	23.0	1838	HIGHWAY	1300.0	2.0	N	THROUGH	WOOD	NaN	S	WOC
5	E7	A	27.0	1840	HIGHWAY	990.0	2.0	N	THROUGH	WOOD	MEDIUM	S	WOC
6	E8	A	28.0	1844	AQUEDUCT	1000.0	1.0	N	THROUGH	IRON	SHORT	S	SUSPE
7	E9	M	3.0	1846	HIGHWAY	1500.0	2.0	N	THROUGH	IRON	SHORT	S	SUSPE
8	E10	A	39.0	1848	AQUEDUCT	1300.0	1.0	N	DECK	WOOD	NaN	S	WOC
9	E11	A	29.0	1851	HIGHWAY	1000.0	2.0	N	THROUGH	WOOD	MEDIUM	S	WOC

In [30]:

```
def rel_l_condition(v):
    if v == "S" or v == "F" or v == "S-F":
        return "no flag"
    else:
        return "flag"
# the apply function was needed here
df['rel_l_flag'] = df['rel-l'].apply(rel_l_condition)
df.head(10)
```

Out[30]:

	Id	river	location	erected	purpose	length	lanes	clear- g	t-or-d	material	span	rel- l	typ
0	E1	M	3.0	1818	HIGHWAY	1300.0	2.0	N	THROUGH	WOOD	SHORT	S	WOC
1	E2	A	25.0	1819	HIGHWAY	1037.0	2.0	N	THROUGH	WOOD	SHORT	S	WOC
2	E3	A	39.0	1829	AQUEDUCT	1300.0	1.0	N	THROUGH	WOOD	NaN	S	WOC
3	E5	A	29.0	1837	HIGHWAY	1000.0	2.0	N	THROUGH	WOOD	SHORT	S	WOC
4	E6	M	23.0	1838	HIGHWAY	1300.0	2.0	N	THROUGH	WOOD	NaN	S	WOC
5	E7	A	27.0	1840	HIGHWAY	990.0	2.0	N	THROUGH	WOOD	MEDIUM	S	WOC
6	E8	A	28.0	1844	AQUEDUCT	1000.0	1.0	N	THROUGH	IRON	SHORT	S	SUSPE
7	E9	M	3.0	1846	HIGHWAY	1500.0	2.0	N	THROUGH	IRON	SHORT	S	SUSPE
8	E10	A	39.0	1848	AQUEDUCT	1300.0	1.0	N	DECK	WOOD	NaN	S	WOC
9	E11	A	29.0	1851	HIGHWAY	1000.0	2.0	N	THROUGH	WOOD	MEDIUM	S	WOC

A good solution is to fill these with MEDIUM

In [31]:

```
# span is missing 16
fill_value = "MEDIUM"
df["span"] = df["span"].fillna(fill_value)
```

Lets look at relations between i. length and purpose ii. lanes and material iii. span and lanes

between length and purpose RR and HIGHWAY are the most common. HIGHWAY is the most recent and most modern data reflecting lengths of 1000 to 4000. 4000 is an outlier, but might be important due to the geography of Pittsburgh and dimensions of the rivers. The earlier materials of the 1800s is WOOD but in recent modern times STEEL has replaced bridge material, lanes have expanded to 2, 4, and 6. These run in two opposite directions, requiring even lanes on each side. MEDIUM and LONG have replaced SHORT and MEDIUM, with span reaching from 1000 up to 3000. Larger values have replaced smaller ones ranging in the 1000s.

```
In [32]: df[["length", "purpose"]].tail(10)
```

```
Out[32]:
```

	length	purpose
98	804.0	HIGHWAY
99	1700.0	HIGHWAY
100	1000.0	HIGHWAY
101	980.0	HIGHWAY
102	2213.0	HIGHWAY
103	870.0	HIGHWAY
104	3756.0	HIGHWAY
105	950.0	HIGHWAY
106	1300.0	HIGHWAY
107	1300.0	HIGHWAY

```
In [33]: df[["lanes", "material"]].tail(10)
```

```
Out[33]:
```

	lanes	material
98	2.0	STEEL
99	2.0	STEEL
100	6.0	STEEL
101	4.0	STEEL
102	4.0	STEEL
103	6.0	STEEL
104	6.0	STEEL
105	6.0	STEEL
106	6.0	STEEL
107	6.0	STEEL

```
In [34]: df[["span", "lanes"]].tail(10)
```

```
Out[34]:
```

	span	lanes
98	MEDIUM	2.0
99	LONG	2.0
100	LONG	6.0
101	MEDIUM	4.0
102	LONG	4.0
103	MEDIUM	6.0
104	LONG	6.0
105	LONG	6.0
106	MEDIUM	6.0
107	MEDIUM	6.0

We should fill these with strings that were common before the year 1884 as S and after 1884 with F.

```
In [35]: # 1. Fill missing values with 'S' where 'erected' is Less than 1884
df.loc[df["erected"] < 1884, "rel-1"] = df["rel-1"].fillna("S")

# 2. Fill the remaining missing values with 'F' (where 'erected' >= 1884)
df.loc[df["erected"] >= 1884, "rel-1"] = df["rel-1"].fillna("F")
```

```
In [36]: df.shape
```

```
Out[36]: (108, 16)
```

The last column is the type and SIMPLE-T is most common and keeps the average under control. 46 which is nearly half are SIMPLE-T

```
In [37]: len(df[df["type"] == "SIMPLE-T"])
```

```
Out[37]: 44
```

```
In [38]: #type is missing 2
fill_value = "SIMPLE-T"
df["type"] = df["type"].fillna(fill_value)
```

```
In [39]: df.shape
```

```
Out[39]: (108, 16)
```

All the missing data has be addressed and next will be located in Cleaning Log.

Step 5: Remove duplicate rows

```
In [40]: print(df.duplicated().sum()) # counts fully repeated rows
df = df.drop_duplicates() # removes them.
```

```
0
```

Duplicated().sum() returns 0 - continue to data type conversion of lanes.

Step 6: Convert the data type of "lanes" to int. - removing noise

```
In [41]: df["lanes"] = df["lanes"].astype(int)
```

All of these should be zero.

```
In [42]: df.isna().sum()
```

```
Out[42]: Id          0
         river       0
         location    0
         erected     0
         purpose     0
         length      0
         lanes       0
         clear-g     0
         t-or-d      0
         material    0
         span        0
         rel-l       0
         type        0
         span_flag   0
         t_or_d_flag 0
         rel_l_flag  0
         dtype: int64
```

Step 7: Since standardizing the date requires more information like day and month, using int works fine for sorting.

```
In [43]: df["erected"]
```

```
Out[43]: 0      1818
         1      1819
         2      1829
         3      1837
         4      1838
         ...
        103     1969
        104     1975
        105     1978
        106     1982
        107     1986
         Name: erected, Length: 108, dtype: int64
```

Step 8: Investigate the quantities

```
In [44]: df.dtypes
```

```
Out[44]: Id          str
         river       str
         location    float64
         erected     int64
         purpose     str
         length      float64
         lanes       int64
         clear-g     str
         t-or-d      str
         material    str
         span        str
         rel-l       str
         type        str
         span_flag   str
         t_or_d_flag str
         rel_l_flag  str
         dtype: object
```

```
In [45]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 108 entries, 0 to 107
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Id              108 non-null   str
1   river           108 non-null   str
2   location        108 non-null   float64
3   erected         108 non-null   int64
4   purpose         108 non-null   str
5   length          108 non-null   float64
6   lanes           108 non-null   int64
7   clear-g         108 non-null   str
8   t-or-d          108 non-null   str
9   material        108 non-null   str
10  span            108 non-null   str
11  rel-l           108 non-null   str
12  type            108 non-null   str
13  span_flag       108 non-null   str
14  t_or_d_flag     108 non-null   str
15  rel_l_flag      108 non-null   str
dtypes: float64(2), int64(2), str(12)
memory usage: 13.6 KB
```

```
In [46]: df.describe()
```

```
Out[46]:
```

	location	erected	length	lanes
count	108.000000	108.000000	108.000000	108.000000
mean	25.987963	1905.314815	1500.601852	2.611111
std	13.602218	37.173664	656.727698	1.190565
min	1.000000	1818.000000	804.000000	1.000000
25%	15.750000	1884.000000	1075.000000	2.000000
50%	27.000000	1903.000000	1300.000000	2.000000
75%	37.250000	1928.000000	1692.500000	4.000000
max	52.000000	1986.000000	4558.000000	6.000000

```
In [47]: df.isna().sum()
```

```
Out[47]: Id          0
         river       0
         location    0
         erected     0
         purpose     0
         length      0
         lanes       0
         clear-g     0
         t-or-d      0
         material    0
         span        0
         rel-l       0
         type        0
         span_flag   0
         t_or_d_flag 0
         rel_l_flag  0
         dtype: int64
```

Step 9: Save the cleaned file

```
In [48]: df.to_csv('Pittsburgh_Bridges.csv', index=False)
```

Step 10: Cleaning Log

- a. All the missing data has been addressed. Each string has been filled.
- b. Noisy data removed, such as floats that can be converted to integers, such as lanes. Avoid spending too much on few data points with obvious values, that can be typed by hand.
- c. Relations shown between: i. length and purpose ii. lanes and materials iii. span and lanes (All addressed in step 4: Impute missing entries).

What has been cleaned:

location: 1 missing replaced with median at 27.

length: 27 missing replaced with median 1300.

lanes: missing 16, replaced with median 2, changed the last most recent entries to 6 lanes, converted to int. to reduce noise.

clear-g: missing 2 strings, #19 filled with N, #20 with G, a nice bisection of data.

t-or-d: missing 6 strings, majority of entries are THROUGH, fill the missing entries with THROUGH, and has a new column called t_or_d_flag indicating the hand typed entries.

material: missing 2 strings, modern dates replaced with STEEL. The most accurate material.

span: missing 16 strings and has a new column called span_flag indicating the hand typed entries, MEDIUM.

rel-l: missing 5 strings, and used a condition, before the year 1884 as S and after 1884 with F, and has a new column called rel_l_flag indicating the hand typed entries.

type: missing 2 strings, NIL is ignored and SIMPLE-T is used for empty strings due to its most common use, however ARCH is most recent.

```
In [49]: df.head(60)
```

Out[49]:

	Id	river	location	erected	purpose	length	lanes	clear- g	t-or-d	material	span	rel- l	
0	E1	M	3.0	1818	HIGHWAY	1300.0	2	N	THROUGH	WOOD	SHORT	S	W
1	E2	A	25.0	1819	HIGHWAY	1037.0	2	N	THROUGH	WOOD	SHORT	S	W
2	E3	A	39.0	1829	AQUEDUCT	1300.0	1	N	THROUGH	WOOD	MEDIUM	S	W
3	E5	A	29.0	1837	HIGHWAY	1000.0	2	N	THROUGH	WOOD	SHORT	S	W
4	E6	M	23.0	1838	HIGHWAY	1300.0	2	N	THROUGH	WOOD	MEDIUM	S	W
5	E7	A	27.0	1840	HIGHWAY	990.0	2	N	THROUGH	WOOD	MEDIUM	S	W
6	E8	A	28.0	1844	AQUEDUCT	1000.0	1	N	THROUGH	IRON	SHORT	S	SU
7	E9	M	3.0	1846	HIGHWAY	1500.0	2	N	THROUGH	IRON	SHORT	S	SU
8	E10	A	39.0	1848	AQUEDUCT	1300.0	1	N	DECK	WOOD	MEDIUM	S	W
9	E11	A	29.0	1851	HIGHWAY	1000.0	2	N	THROUGH	WOOD	MEDIUM	S	W
10	E12	A	39.0	1853	RR	1300.0	2	N	DECK	WOOD	MEDIUM	S	W
11	E14	M	6.0	1856	HIGHWAY	1200.0	2	N	THROUGH	WOOD	MEDIUM	S	W
12	E13	A	33.0	1856	HIGHWAY	1300.0	2	N	THROUGH	WOOD	MEDIUM	S	W
13	E15	A	28.0	1857	RR	1300.0	2	N	THROUGH	WOOD	MEDIUM	S	W
14	E16	A	25.0	1859	HIGHWAY	1030.0	2	N	THROUGH	IRON	MEDIUM	S-F	SU
15	E17	M	4.0	1863	RR	1000.0	2	N	THROUGH	IRON	MEDIUM	S	SIMI
16	E18	A	28.0	1864	RR	1200.0	2	N	THROUGH	IRON	SHORT	S	SIMI
17	E19	A	29.0	1866	HIGHWAY	1000.0	2	N	THROUGH	WOOD	MEDIUM	S	W
18	E20	A	32.0	1870	HIGHWAY	1000.0	2	N	THROUGH	WOOD	MEDIUM	S	W
19	E21	M	16.0	1874	RR	1300.0	2	N	THROUGH	IRON	MEDIUM	S	SIMI
20	E23	M	1.0	1876	HIGHWAY	1245.0	2	G	THROUGH	STEEL	LONG	F	SU
21	E22	A	24.0	1876	HIGHWAY	1200.0	4	G	THROUGH	WOOD	SHORT	S	W
22	E24	O	45.0	1878	RR	1300.0	2	G	THROUGH	STEEL	MEDIUM	S	SIMI
23	E25	M	10.0	1882	RR	1300.0	2	G	THROUGH	STEEL	MEDIUM	S	SIMI
24	E27	A	39.0	1883	RR	1300.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI
25	E26	M	12.0	1883	RR	1150.0	2	G	THROUGH	STEEL	MEDIUM	S	SIMI
26	E30	A	31.0	1884	RR	1300.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI
27	E29	A	26.0	1884	HIGHWAY	1080.0	2	G	THROUGH	STEEL	MEDIUM	F	SU
28	E28	M	3.0	1884	HIGHWAY	1000.0	2	G	THROUGH	STEEL	MEDIUM	S	A
29	E32	A	30.0	1887	HIGHWAY	1300.0	2	G	THROUGH	IRON	MEDIUM	F	SIMI
30	E31	M	8.0	1887	RR	1161.0	2	G	THROUGH	STEEL	MEDIUM	S	SIMI
31	E34	O	41.0	1888	RR	4558.0	2	G	THROUGH	STEEL	LONG	F	SIMI
32	E33	M	19.0	1889	HIGHWAY	1120.0	2	G	THROUGH	IRON	MEDIUM	F	SIMI
33	E36	O	45.0	1890	HIGHWAY	1300.0	2	G	THROUGH	IRON	SHORT	F	SIMI
34	E35	A	27.0	1890	HIGHWAY	1000.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI

	Id	river	location	erected	purpose	length	lanes	clear- g	t-or-d	material	span	rel- l	
35	E38	M	17.0	1891	HIGHWAY	1300.0	2	G	THROUGH	IRON	MEDIUM	F	SIMI
36	E37	M	18.0	1891	RR	1350.0	2	G	THROUGH	STEEL	MEDIUM	S	SIMI
37	E39	A	25.0	1892	HIGHWAY	1300.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI
38	E4	A	27.0	1892	AQUEDUCT	1092.0	1	N	THROUGH	WOOD	SHORT	S	W
39	E40	M	22.0	1893	HIGHWAY	1300.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI
40	E41	M	11.0	1894	HIGHWAY	1300.0	2	G	THROUGH	IRON	MEDIUM	F	SIMI
41	E42	M	9.0	1895	HIGHWAY	2367.0	2	G	THROUGH	STEEL	LONG	F	SIMI
42	E44	O	48.0	1896	HIGHWAY	1300.0	2	G	THROUGH	STEEL	LONG	F	SU
43	E43	M	7.0	1896	HIGHWAY	1040.0	2	G	THROUGH	STEEL	LONG	F	A
44	E46	A	37.0	1897	RR	4000.0	2	G	DECK	STEEL	LONG	F	SIMI
45	E45	M	14.0	1897	RR	2264.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI
46	E47	M	15.0	1898	RR	2000.0	2	G	THROUGH	STEEL	MEDIUM	S	SIMI
47	E58	A	33.0	1900	HIGHWAY	1200.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI
48	E48	A	38.0	1900	HIGHWAY	2000.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI
49	E94	M	13.0	1901	RR	1300.0	2	G	THROUGH	STEEL	LONG	F	SIMI
50	E49	A	34.0	1902	HIGHWAY	1850.0	2	G	THROUGH	STEEL	MEDIUM	F	CAN'
51	E95	M	16.0	1903	RR	1300.0	2	G	THROUGH	STEEL	MEDIUM	S	SIMI
52	E87	A	35.0	1903	RR	3000.0	2	G	THROUGH	STEEL	MEDIUM	S	SIMI
53	E51	M	6.0	1903	RR	1417.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI
54	E50	M	21.0	1903	RR	1154.0	2	G	THROUGH	STEEL	LONG	F	SIMI
55	E89	M	4.0	1904	RR	1200.0	2	G	THROUGH	STEEL	MEDIUM	S-F	SIMI
56	E53	A	28.0	1904	RR	965.0	4	G	THROUGH	STEEL	MEDIUM	S-F	SIMI
57	E52	M	2.0	1904	RR	1504.0	2	G	THROUGH	STEEL	LONG	F	CAN'
58	E54	Y	27.0	1908	HIGHWAY	1240.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI
59	E56	M	23.0	1909	HIGHWAY	1300.0	2	G	THROUGH	STEEL	MEDIUM	F	SIMI

End Of Lab- Behzad Ghabaei

In []: